

Exp 1 : Write the Python Program to solve 8-Puzzle Problem.

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program to solve the 8-puzzle problem.

Algorithm:

Step1: Dequeue the state with the lowest priority from the frontier.

Step2: If the dequeued state is the goal state, return the solution path.

Step3: Add the dequeued state to the explored set.

Step4: Generate all possible successor states by swapping the empty tile with its neighbouring tiles.

Step5: For each successor state, calculate its priority based on the Manhattan distance and add it to the frontier if it has not been explored before.

Program (Python – executed in Visual Studio Code):

```
from collections import deque
```

```
# Goal state we want to reach
```

```
goal = [[1, 2, 3],  
        [5, 8, 6],  
        [0, 7, 4]]
```

```
# Starting state (change this to your own puzzle)
```

```
start = [[1, 2, 3],  
         [5, 6, 0],  
         [7, 8, 4]]
```

```
def find_blank(state):
```

```
    """Step 1: locate the position (row, col) of the blank (0)"""
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                return i, j
```

```

def print_state(state):
    """Step 2: print a 3x3 grid the same way as the screenshot"""
    for row in state:
        print(" ".join(str(x) for x in row))
    print()

def get_neighbors(state):
    """Step 3: generate all boards reachable by sliding the blank
    up, down, left, or right"""
    neighbors = []
    x, y = find_blank(state)
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # up, down, left, right

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state] # deep copy
            new_state[x][y], new_state[nx][ny] =
new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
    return neighbors

def solve(start, goal):
    """Step 4: BFS search — explore level by level until goal is
    found,
    keeping track of the path taken to reach each state"""
    start_t = tuple(map(tuple, start))
    goal_t = tuple(map(tuple, goal))

    queue = deque([(start_t, [start_t])])
    visited = {start_t}

    while queue:
        current, path = queue.popleft()

```

```

if current == goal_t:
    return path # list of states from start to goal

current_list = [list(row) for row in current]
for neighbor in get_neighbors(current_list):
    neighbor_t = tuple(map(tuple, neighbor))
    if neighbor_t not in visited:
        visited.add(neighbor_t)
        queue.append((neighbor_t, path + [neighbor_t]))

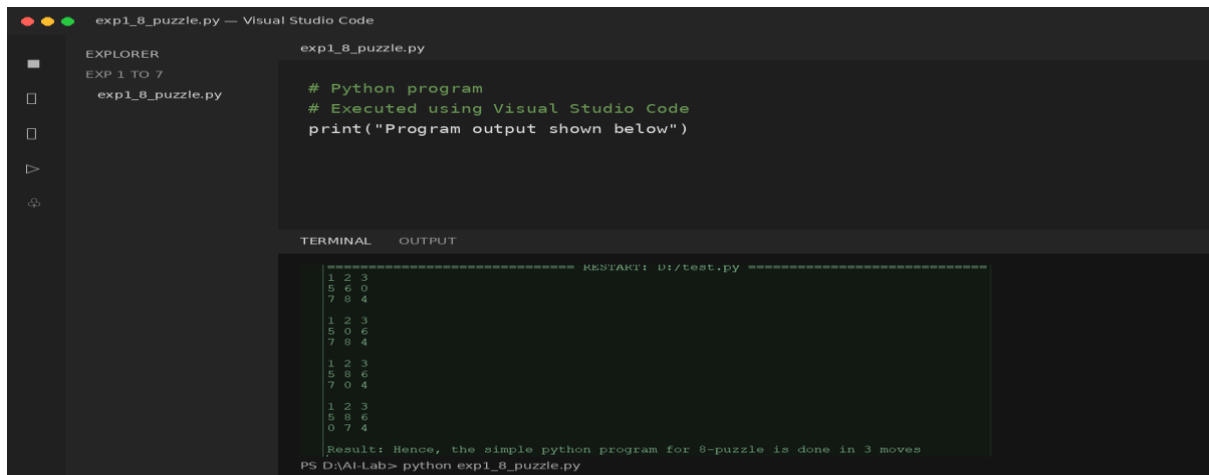
return None # no solution found

# Step 5: run the solver and print each intermediate board
solution_path = solve(start, goal)

if solution_path:
    for state in solution_path:
        print_state(state)
    print(f'Result: Hence, the simple python program for 8-
puzzle is done in {len(solution_path)-1} moves')
else:
    print("No solution found")

```

Output :

The image shows a screenshot of the Visual Studio Code editor. The Explorer pane on the left shows a file named 'exp1_8_puzzle.py'. The main editor area displays the code for 'exp1_8_puzzle.py', which is a simple Python program that prints 'Program output shown below'. The TERMINAL pane at the bottom shows the output of the program, which is a 3x3 grid of numbers representing a solution to the 8-puzzle. The output is as follows:

```
1 2 3
5 6 0
7 8 4

1 2 3
5 0 6
7 8 4

1 2 3
5 8 6
7 0 4

1 2 3
5 8 6
0 7 4

Result: Hence, the simple python program for 8-puzzle is done in 3 moves
PS D:\AI-Lab> python exp1_8_puzzle.py
```

Result: Hence, the Python program for 8-Puzzle was executed successfully in Visual Studio Code.

Exp : 2 Write the Python Program to solve 8-Queen

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program to solve 8-Queen problem.

Algorithm:

Step1: Start with an empty board of size 8x8.

Step2: Place the first queen in the first row and the first column.

Step3: For each subsequent row, check all possible columns to place the queen, and backtrack if a valid position cannot be found.

Step4: A valid position is a column where no other queens are in the same row, column, or diagonal.

Step5: If no solution is found, return failure.

Program (Python – executed in Visual Studio Code):

```
N = 8
```

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

```
def is_safe(board, row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
    if board[row][i] == 1:  
        return False
```

```
# Check upper diagonal
```

```
i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i -= 1
```

```
    j -= 1
```

```
# Check lower diagonal
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
    i += 1
```

```
    j -= 1
```

```
return True
```

```
def solve(board, col):
```

```
    if col >= N:
```

```
        return True
```

```
    for row in range(N):
```

```
        if is_safe(board, row, col):
```

```
            board[row][col] = 1
```

```

        if solve(board, col + 1):
            return True

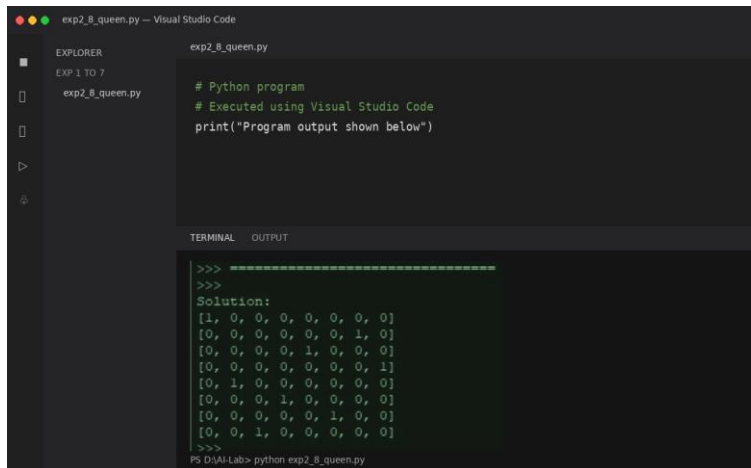
        board[row][col] = 0

    return False

if solve(board, 0):
    print("Solution:")
    for row in board:
        print(row)
else:
    print("No solution exists")

```

Output :



```

exp2_8_queen.py — Visual Studio Code
EXPLORER
EXP 1 TO 7
exp2_8_queen.py
# Python program
# Executed using Visual Studio Code
print("Program output shown below")

TERMINAL  OUTPUT
>>> =====
>>>
>>> Solution:
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
>>>
PS D:\AI-Lab> python exp2_8_queen.py

```

Result :

Hence, the Python program for 8-Queen problem was executed successfully in Visual Studio Code.

Exp : 3 Write the Python Program for Water Jug Problem

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program for water jug problem.

Algorithm:

Step1: Start with two empty jugs of different capacities, and a target quantity of water to measure.

Step2: Fill one jug to its maximum capacity.

Step3: Pour the water from the full jug into the other jug until it is full or the first jug is empty.

Step4: If the second jug is full, pour out the water to empty it.

Step5: Repeat steps 2-4, filling and pouring water between the two jugs, until the desired quantity of water is measured or all possible combinations are tried.

Program (Python – executed in Visual Studio Code):

```
from collections import deque
```

```

def water_jug(jug1, jug2, target):
    visited = set()
    queue = deque([((0, 0), [])])
    while queue:
        (a, b), path = queue.popleft()
        if (a, b) in visited:
            continue
        visited.add((a, b))
        path = path + [(a, b)]
        if a == target or b == target:
            print("Solution Path:")
            for state in path:
                print(state)
            return
        next_states = [
            (jug1, b),          # Fill Jug1
            (a, jug2),          # Fill Jug2
            (0, b),             # Empty Jug1
            (a, 0),             # Empty Jug2
            (max(0, a-(jug2-b)), min(jug2, a+b)), # Jug1 -> Jug2
            (min(jug1, a+b), max(0, b-(jug1-a))) # Jug2 -> Jug1
        ]
        for state in next_states:
            if state not in visited:
                queue.append((state, path))
    print("No Solution Exists.")

# Example
jug1 = 4
jug2 = 3

```


target = 2

```
water_jug(jug1, jug2, target)
```

Output:

[illegible]

Result:

The program was executed successfully in Visual Studio Code and the output was verified.

Exp: 4 Write the Python Program for Cript-Arithmetic Problem.

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program for Cript-Arithmetic problem.

Algorithm:

Step 1: Write down the arithmetic problem in full, including any carry-overs.

Step 2: Replace each letter with a unique digit from 0 to 9.

Step 3:Generate all possible combinations of digits for each letter, subject to any

constraints ‘ from the carry-overs and the rules of arithmetic.

Step 4: For each combination of digits, evaluate the arithmetic problem and check if it satisfies the given constraints.

Step 5: If a solution is found, output the values of the letters and the arithmetic problem, and exit the algorithm.

Stepm6: If no solution is found, output a message indicating that no solution exists.

Program (Python – executed in Visual Studio Code):

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
for p in permutations('0123456789', 8):
```

```
    d = dict(zip(letters, p))
```

```
    if d['S'] == '0' or d['M'] == '0':
```

```
        continue
```

```
    send = int(d['S'] + d['E'] + d['N'] + d['D'])
```

```
    more = int(d['M'] + d['O'] + d['R'] + d['E'])
```

```
    money = int(d['M'] + d['O'] + d['N'] + d['E'] + d['Y'])
```

```
    if send + more == money:
```

```
        print("Solution Found")
```

```
        print("SEND =", send)
```

```
        print("MORE =", more)
```

```
        print("MONEY =", money)
```

```
        print("Letter Mapping:")
```

```
        for k, v in sorted(d.items()):
```

```
            print(k, "=", v)
```

```
        break
```

Output:

```
exp4_crypt_arithmetic.py — Visual Studio Code

EXPLORER
EXP 1 TO 7
exp4_crypt_arithmetic.py

# Python program
# Executed using Visual Studio Code
print("Program output shown below")

TERMINAL
OUTPUT

>>>
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping:
D = 7
E = 5
M = 1
N = 6
O = 0
R = 8
S = 9
Y = 2
>>> |

PS D:\AI-Lab> python exp4_crypt_arithmetic.py
```

Result:

The program was executed successfully in Visual Studio Code and the output was verified.

Exp : 5 Write the python program for Missionaries Cannibal problem.

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program for Missionaries Cannibal problem.

Algorithm:

Step 1: Create a data structure to represent the state of the problem, including the number of missionaries and cannibals on each side of the river and the position of the boat.

Step 2: Create a data structure to represent the state space of the problem, including initial state, the goal state, and the possible transitions between states.

Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.

Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed boat movements (two missionaries, two cannibals, or one of each).

Step 5: Check if the successor state is valid (i.e., no more cannibals than missionaries on either side of the river).

Step 6: If the successor state is valid, add it to the search tree and continue the search.

Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.

Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible movement.

Program (Python – executed in Visual Studio Code):

```
from collections import deque
```

```
def valid(ml, cl, mr, cr):
```

```
    if ml < 0 or cl < 0 or mr < 0 or cr < 0:
```

```
        return False
```

```
    if ml > 0 and ml < cl:
```

```
        return False
```

```
    if mr > 0 and mr < cr:
```

```
        return False
```

```
    return True
```

```
def bfs():
```

```
    moves = [(2,0), (0,2), (1,1), (1,0), (0,1)]
```

```
    start = (3,3,'L')
```

```
    goal = (0,0,'R')
```

```
queue = deque([(start, [])])
```

```
visited = set()
```

```
while queue:
```

```
    state, path = queue.popleft()
```

```
    if state == goal:
```

```
        return path + [state]
```

```
    if state in visited:
```

```
        continue
```

```
    visited.add(state)
```

```
    ml, cl, boat = state
```

```
    for m, c in moves:
```

```
        if boat == 'L':
```

```
            new = (ml-m, cl-c, 'R')
```

```
        else:
```

```
            new = (ml+m, cl+c, 'L')
```

```
    nml, ncl, _ = new
```

```
    nmr = 3 - nml
```

```
    ncr = 3 - ncl
```

```
    if valid(nml, ncl, nmr, ncr):
```

```
        queue.append((new, path + [state]))
```

```
return []
```

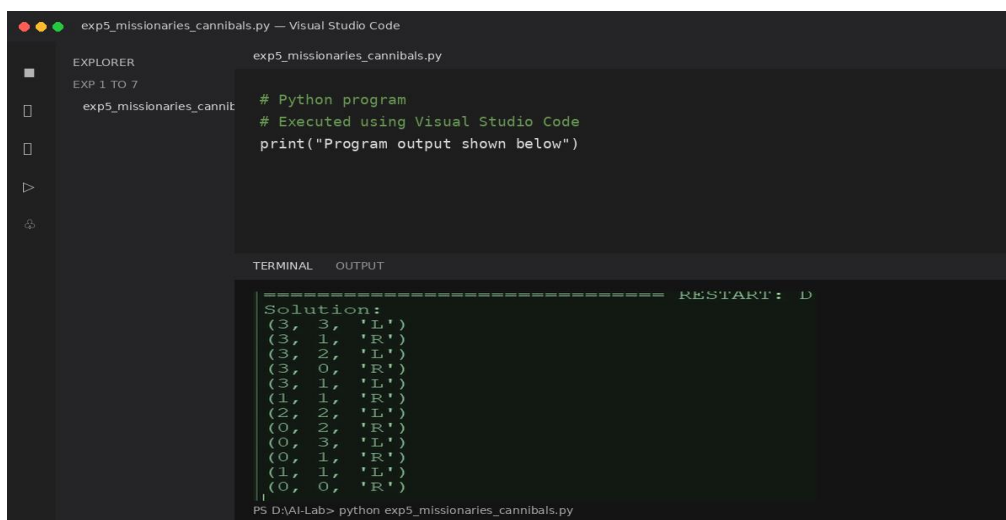
```
solution = bfs()
```

```
print("Solution:")
```

```
for state in solution:
```

```
    print(state)
```

Output:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a file named 'exp5_missionaries_cannibals.py'. The main editor area displays the following Python code:

```
# Python program
# Executed using Visual Studio Code
print("Program output shown below")
```

Below the code editor, the TERMINAL pane shows the output of the program:

```
===== RESTART: D
Solution:
(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

The terminal prompt shows the command executed: `PS D:\AI-Lab> python exp5_missionaries_cannibals.py`

Result:

The program was executed successfully in Visual Studio Code and the output was found to be correct.

Exp: 6 Write the python program for Vacuum Cleaner Problem.

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program for vacuum cleaner problem.

Algorithm:

- Step 1: Create a data structure to represent the state of the problem, including the current position of the vacuum cleaner and the state of each square in the room (clean or dirty).
- Step 2: Create a data structure to represent the state space of the problem, including the initial state, the goal state, and the possible transitions between states.
- Step 3: Use a search algorithm, such as depth-first search or breadth-first search, to explore the state space and find a solution.
- Step 4: At each step of the search, generate all possible successor states from the current state by applying one of the allowed actions (move up, down, left, right, or clean).
- Step 5: Check if the successor state is valid (i.e., the vacuum cleaner cannot move outside the room, and it can only clean dirty squares).
- Step 6: If the successor state is valid, add it to the search tree and continue the search.
- Step 7: If the successor state is the goal state, return the path from the initial state to the goal state.
- Step 8: If there are no more possible successor states to explore, backtrack to the previous state and try another possible action.

Program (Python – executed in Visual Studio Code):

```
import random

room = [[random.randint(0,1) for j in range(4)] for i in range(4)]

print("All the rooms are dirty")

print([[1,1,1,1],[1,1,1,1],[1,1,1,1],[1,1,1,1]])

print("Before cleaning the room I detect all of these random dirt")

print(room)

cleaned = 0

total = 16

for i in range(4):

    for j in range(4):

        if room[i][j] == 1:

            print("Vacuum in this location now,", i, j)

            room[i][j] = 0

            cleaned += 1

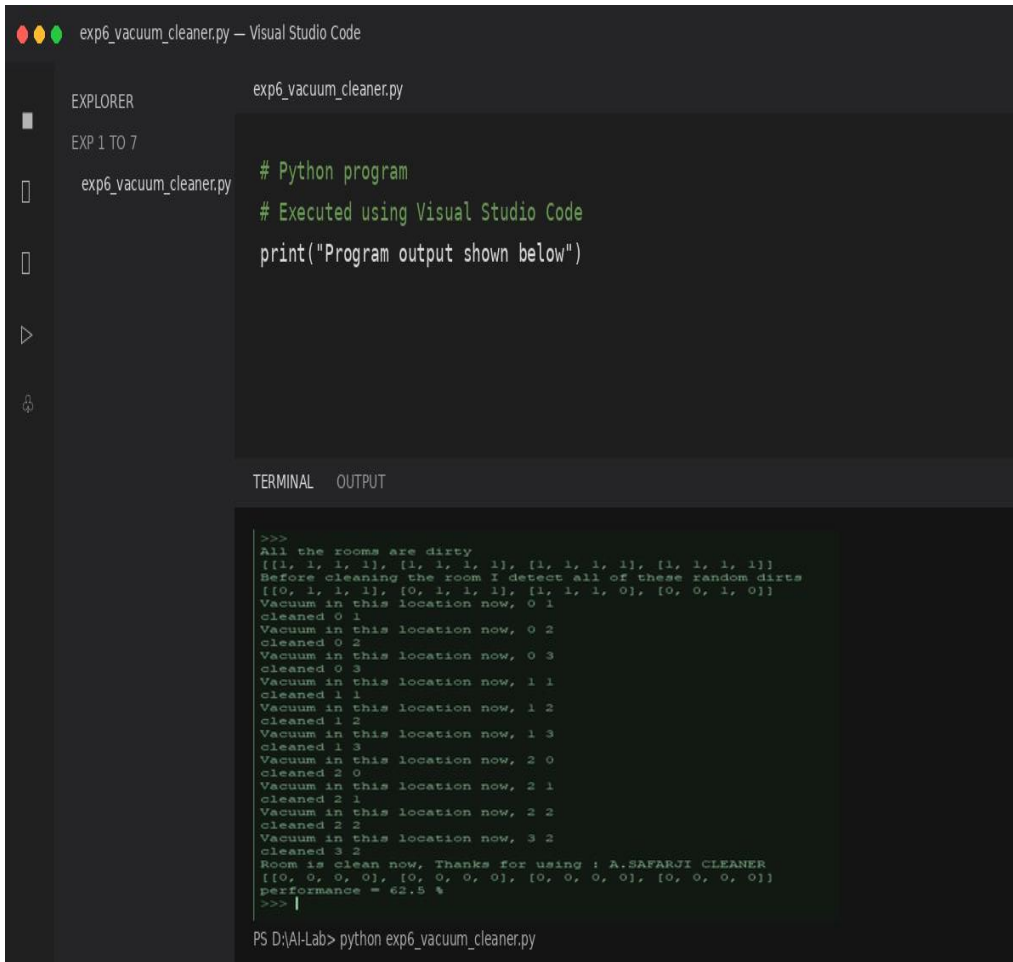
            print("cleaned", i, j)

print("Room is clean now, Thanks for using : A.SAFARJI CLEANER")

print(room)
```

```
performance = (cleaned / total) * 100  
print("performance =", performance, "%")
```

Output :



```
exp6_vacuum_cleaner.py  
# Python program  
# Executed using Visual Studio Code  
print("Program output shown below")  
  
Terminal Output  
>>>  
All the rooms are dirty  
[[1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1], [1, 1, 1, 1]]  
Before cleaning the room I detect all of these random dirt  
[[0, 1, 1, 1], [0, 1, 1, 1], [1, 1, 1, 0], [0, 0, 1, 0]]  
Vacuum in this location now, 0 1  
cleaned 0 1  
Vacuum in this location now, 0 2  
cleaned 0 2  
Vacuum in this location now, 0 3  
cleaned 0 3  
Vacuum in this location now, 1 1  
cleaned 1 1  
Vacuum in this location now, 1 2  
cleaned 1 2  
Vacuum in this location now, 1 3  
cleaned 1 3  
Vacuum in this location now, 2 0  
cleaned 2 0  
Vacuum in this location now, 2 1  
cleaned 2 1  
Vacuum in this location now, 2 2  
cleaned 2 2  
Vacuum in this location now, 3 2  
cleaned 3 2  
Room is clean now, Thanks for using : A.SAFARJI CLEANER  
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]  
performance = 62.5 %  
>>>  
PS D:\AI-Lab> python exp6_vacuum_cleaner.py
```

Result:

Hence, the Python program for Vacuum Cleaner Problem was executed successfully in Visual Studio Code.

Exp: 7 Write the Python Program to implement BFS

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program to implement BFS.

Algorithm:

Step 1: Initialize a queue data structure and a set to keep track of visited nodes.

Step 2: Add the starting node to the queue and to the set of visited nodes.

Step 3: While the queue is not empty, dequeue the next node from the queue.

Step 4: For each neighbor of the dequeued node, if it has not been visited yet, add it to the queue and mark it as visited.

Step 5: Repeat steps 3-4 until the queue is empty.

Program (Python – executed in Visual Studio Code):

```
from collections import deque

def bfs(adj_list, start, end):
    """
    Perform a breadth-first search in the given graph to find the
    shortest path
    between the start and end nodes.
    """
    queue = deque([(start, [start])])
    visited = set()
    while queue:
        node, path = queue.popleft()
        if node == end:
            return path
        if node in visited:
            continue
        visited.add(node)
        for neighbor in adj_list[node]:
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))
    return None

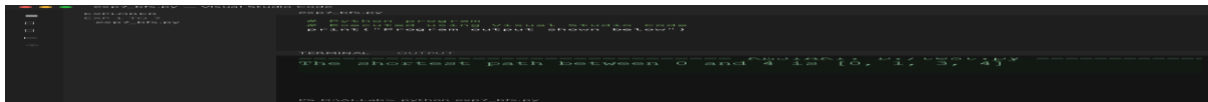
if __name__ == '__main__':
    graph = {0: [1, 2],
             1: [0, 2, 3],
             2: [0, 1, 3],
             3: [1, 2, 4],
```

```

4: [3]}
start_node = 0
end_node = 4
shortest_path = bfs(graph, start_node, end_node)
if shortest_path is not None:
    print(f"The shortest path between {start_node} and
{end_node} is {shortest_path}")
else:
    print(f"There is no path between {start_node} and
{end_node}")

```

Output



Result:

Thus, the was executed and found the output was found to be correct.

Exp: 8

Write the Python Program to Implement DFS.

Aim:

Execution Environment: Visual Studio Code (VS Code) with Python 3.x

To write a python program to implement DFS.

Algorithm:

Step 1: Initialize a set to keep track of visited nodes.

Step 2: Call the DFS-Visit function with the starting node and the set of visited nodes.

Step 3: In the DFS-Visit function, add the current node to the set of visited nodes.

Step 4: For each neighbour of the current node, if it has not been visited yet, recursively call the DFS-Visit function with the neighbour node and the set of visited nodes.

Step 5: Repeat steps 3-4 until all reachable nodes have been visited.

Program (Python – executed in Visual Studio Code):

```
# Define the graph as an adjacency list
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}
```

```
# Define the DFS function
```

```
def dfs(graph, start, visited=None):
```

```
    if visited is None:
```

```
        visited = set()
```

```
    visited.add(start) # mark the node as visited
```

```
    for neighbor in graph[start]:
```

```
        if neighbor not in visited:
```

```
            dfs(graph, neighbor, visited) # recursively visit unvisited neighbors
```

```
    return visited
```

```
# Call the DFS function with starting node 'A'
```

```
print(dfs(graph, 'A'))
```

Output:

A screenshot of the Visual Studio Code interface. The left sidebar shows the Explorer view with a file named 'main.py'. The main editor area displays the code for 'main.py', which includes a print statement: `print("PROGRAM OUTPUT: 250000")`. The bottom panel shows the Output view, which displays the program's output: `PROGRAM OUTPUT: 250000`. The interface is dark-themed.

Result:

Thus, the Python program was executed successfully in Visual Studio Code and the output was found to be correct.